// 25 SHORT LESSONS · C++ → UNREAL ENGINE 5.8

UNREAL 101 Cheat-Sheet

Type it. Break it. Fix it. Ship it.



WORLD 1

Terminal Town: C++ basics



THE LOOP

Build and run

```
#include <iostream>
int main() {
  std::cout << "Hi, Orby!\n";
  return 0;
}
```

cl /std:c++20 /W4 /EHsc main.cpp

TYPES

Every value has a type

```
int hp = 100;
double speed = 2.5;
bool alive = true;
std::string name = "Orby";
const int MAX_HP = 100;
auto xp = 42;
```

C++ checks types BEFORE you run.

FLOW

Decide and loop

```
if (hp <= 0) {
  alive = false;
} else if (hp < 20) {
  std::cout << "Low!\n";
}
while (alive) { /* a turn */ }
for (int i = 0; i < 3; ++i) { }
```

FUNCTIONS

Functions

```
int RollDice(int sides) {
  return 1 + rand() % sides;
}
void Say(std::string m); // declare
```

VECTOR

Lists that know their type

```
std::vector<int> bag = {5, 8};
bag.push_back(13);
bag.size();        // 3
for (int item : bag) { }
```

STRUCT

Your own types

```
struct Fighter {
  std::string name;
  int hp = 10;
};
Fighter g{"Goblin", 8};
g.hp -= 3;
```

CLASS

Classes guard their data

```
class Hero {
public:
  Hero(int hp) : hp_(hp) {}
  int Hp() const { return hp_; }
private:
  int hp_;
};
```

REFERENCES · MUST MEMORIZE

Copy or original?

```
void Heal(Fighter& f) {
  f.hp += 5;  // the original
}
void Show(const Fighter& f);
  // read-only, no copy
void Oops(Fighter f) { }
  // a COPY!
```

JS shares objects. C++ COPIES!

POINTERS

Pointers, honestly

```
Fighter* t = &g;  // address of g
t->hp = 5;        // (*t).hp
Fighter* none = nullptr;
if (t != nullptr) { }
```

JS: every object var is secretly one

Never use a pointer after its object is gone.

VIRTUAL

One interface, many monsters

```
struct Monster {
  virtual ~Monster() = default;
  virtual void Attack() = 0;
};
struct Goblin : Monster {
  void Attack() override { }
};
```

HEADERS

Many files

```
// Hero.h
#pragma once
class Hero {
public: int Hp() const; };

// Hero.cpp
#include "Hero.h"
int Hero::Hp() const { return 7; }
```

.h declares. .cpp defines.

UNIQUE_PTR

Who owns this?

```
auto m =
  std::make_unique<Goblin>();
m->Attack();
std::vector<
  std::unique_ptr<Monster>> horde;
// freed automatically. No delete!
```

JS: garbage collector does it

One owner. Cleanup is automatic.



The compiler is a strict teacher.
Read the FIRST error only.



WORLD 2

Orb Arena: C++ in Unreal Engine

LOG + TYPES

Say it in the log

```
UE_LOG(LogTemp, Warning,
  TEXT("HP=%d %s"), Hp, *Name);
int32  float  FString  FName  FText
```

TEXT() on literals. A * before an FString!

ACTOR · MUST MEMORIZE

Your first Actor

```
UCLASS()
class AOrb : public AActor {
  GENERATED_BODY()
public:
  AOrb();
  void BeginPlay() override;
  void Tick(float Dt) override;
};
```

Call Super::Tick(Dt) first. #include .generated.h LAST.

UPROPERTY + UFUNCTION

Knobs for the editor

```
UPROPERTY(EditAnywhere,
  BlueprintReadWrite,
  Category="Orb")
float Spin = 90.f;
UFUNCTION(BlueprintCallable)
void Collect();
```

COMPONENTS

Actors are built from parts

```
UPROPERTY(VisibleAnywhere)
TObjectPtr<UStaticMeshComponent>
  Mesh;
// in the constructor:
Mesh = CreateDefaultSubobject<
  UStaticMeshComponent>(
  TEXT("Mesh"));
RootComponent = Mesh;
Light->SetupAttachment(Mesh);
```

FVECTOR

Move and spin

```
FVector P = GetActorLocation();
P.Z += 50.f;
SetActorLocation(P);
AddActorLocalRotation(
  FRotator(0.f, Spin * Dt, 0.f));
```

X forward, Y right, Z up.

TARRAY + TMAP

Unreal's containers

```
TArray<int32> Scores;
Scores.Add(10);
Scores.Num();        // size
for (int32 S : Scores) { }
TMap<FString, int32> Pts;
Pts.Add(TEXT("gold"), 50);
```

JS: array and Map

OVERLAP + CAST

Something touched me

```
Sphere->OnComponentBeginOverlap
  .AddDynamic(this, &AOrb::OnHit);
UFUNCTION()   // required!
void OnHit(UPrimitiveComponent* C,
  AActor* Other,
  UPrimitiveComponent* OC,
  int32 Idx, bool bSweep,
  const FHitResult& Hit);
if (ACharacter* P =
    Cast<ACharacter>(Other)) { }
```

SPAWN + TIMER

Things appear over time

```
UPROPERTY(EditAnywhere)
TSubclassOf<AOrb> OrbClass;
GetWorld()->SpawnActor<AOrb>(
  OrbClass, Loc,
  FRotator::ZeroRotator);
FTimerHandle T;
GetWorldTimerManager().SetTimer(
  T, this, &AGM::Spawn, 2.f, true);
```

BLUEPRINT BRIDGE

C++ and Blueprints

```
UFUNCTION(BlueprintCallable)
  // Blueprint calls your C++
UFUNCTION(
  BlueprintImplementableEvent)
  // C++ calls a Blueprint
UFUNCTION(BlueprintNativeEvent)
  // C++ default, BP overrides
```

GARBAGE COLLECTION · MUST MEMORIZE

Memory the Unreal way

```
UPROPERTY()
AActor* Keep;   // GC-safe
TWeakObjectPtr<AActor> Maybe;
if (Maybe.IsValid()) { }
if (IsValid(Ptr)) { }
Destroy();   // remove an Actor
```

A raw UObject pointer without UPROPERTY() can dangle.

DELEGATES

Your own events

```
DECLARE_DYNAMIC_MULTICAST_DELEGATE_OneParam(
  FOnScore, int32, NewScore);
UPROPERTY(BlueprintAssignable)
FOnScore OnScore;
OnScore.Broadcast(Score);
GM->OnScore.AddDynamic(
  this, &AHud::Show);
```

JS: addEventListener

NAME TAGS

The first letter tells you what it is

A	Actor	AOrb
U	Object / Component	UStaticMeshComponent
F	struct / value type	FVector
E	enum	EOrbType
T	template / container	TArray
I	interface	IInteractable
b	bool variable	bAlive

Boss fights: common bugs

Compiler yelling? Read the FIRST error only. Fix it. Rebuild.

Copy trap No & means a copy. Use Fighter& to change the original.

Missing semicolon A class or struct ends with }; (the semicolon!)

Log shows garbage FString in UE_LOG needs a * in front.

Editor ignores your handler AddDynamic functions need UFUNCTION().

Crash on a dead Actor Keep UObject pointers in UPROPERTY() or use TWeakObjectPtr.